

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



NATURAL LANGUAGE PROCESSING (CO3085)

Assignment

“Duplicate question detection”

Instructor(s): Nguyễn Đức Dũng

Students: Phạm Trần Minh Trí - 2313622 (*Group TN01*)

HO CHI MINH CITY, JUNE 2026



Contents

List of Figures	3
List of Tables	3
1 Introduction	3
2 Problem and dataset	4
2.1 Problem description	4
2.2 Dataset EDA	4
3 Literature review	7
3.1 BERT	7
3.2 SBERT	8
4 Training	10
4.1 BERT	10
4.2 SBERT	11
4.2.1 Approach 1: Cosine similarity - no training	12
4.2.2 Approach 2: Logistic classifier - training	13
5 Result and insight	14
6 Conclusion	17
References	18



List of Figures

1	Quora related questions	4
2	Histograms	5
3	Word cloud	6
4	BERT training	7
5	SBERT inference	9
6	BERT training graph	15
7	SBERT training graph	16

List of Tables

1	Sample records from the Quora Question Pairs dataset.	5
2	GLUE results	8
3	Performance comparison of BERT and SBERT models	14



1 Introduction

2 Problem and dataset

2.1 Problem description

In many online question forum, such as Quora, StackOverflow, or Reddit, where people post a question and other people post their answer, it's common that it's a question that has been asked before with different wordings. For example, 'What is Segmentation fault?' is the same as 'Can somebody please explain to me what segfault is?? I keep getting this error.' It would be helpful to show the asker the answers in the similar post, to not waste time trying to answer the same question over and over again.

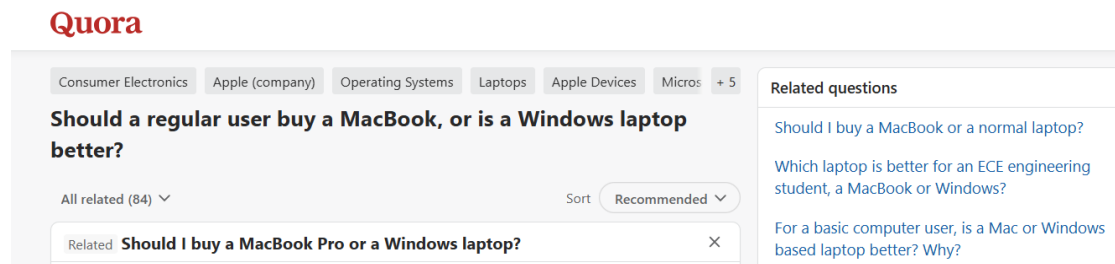


Figure 1: Quora related questions

This is a problem of 'Semantic similarity', in this case it's specifically 'duplicate question detection'. We are trying to determine whether 2 question has the same, or close enough, semantic meaning. In the scope of this project, we will work with the dataset 'Quora question pairs', with each sample containing 2 questions and a label telling whether those 2 are duplicated or not.

Historically, processing natural language has always been a complex and tedious task with countless rules and constraints, as languages are always evolving and new words, phrases, meaning keep changing over time, so much that ordinary grammar rules struggle to keep up with. In recent years, thanks to the development of computing and modern deep learning architecture, especially Transformer, the approach to these problem has been more friendly, as we move into extracting embedding vector to represent the meaning of natural language. In this project, we will be looking at the 2 commonly used model for this task, which is BERT and SBERT.

2.2 Dataset EDA

First, we need to understand the dataset we are about to work with.

We download the dataset from Kaggle [6], then use Python library `pandas` to load the csv file to memory.

We try printing the first 10 rows from the dataset, shown in table 1. The important bits are the text of question 1, question 2, and the label indicating whether they are duplicated. Two sentence may look similar but can mean entirely different things.

After that, we clean the null values from the table, and display some information about the table. We see the dataset has 404348 samples, with 149306 positive and 255042 negative samples. This mean the dataset is quite imbalance, hence during evaluation we will use metrics like precision, recall, f1 score, together with accuracy.

We also do some basic plotting, with the histogram of character counts and words count, plus the word cloud showing the most appearing words in our dataset.

ID	QID1	QID2	Question 1	Question 2	Duplicate
0	1	2	What is the step by step guide to invest in share market in India?	What is the step by step guide to invest in share market?	0
1	3	4	What is the story of Kohinoor (Koh-i-Noor) Diamond?	What would happen if the Indian government stole the Kohinoor (Koh-i-Noor) diamond back?	0
2	5	6	How can I increase the speed of my internet connection while using a VPN?	How can Internet speed be increased by hacking through DNS?	0
3	7	8	Why am I mentally very lonely? How can I solve it?	Find the remainder when 23^{24} is divided by 24,23?	0
4	9	10	Which one dissolve in water quickly: sugar, salt, methane and carbon dioxide?	Which fish would survive in salt water?	0
5	11	12	Astrology: I am a Capricorn Sun, Cap moon and Cap rising... what does that say about me?	I'm a triple Capricorn (Sun, Moon and ascendant in Capricorn). What does this say about me?	1
6	13	14	Should I buy Tiago?	What keeps children active and far from phone and video games?	0
7	15	16	How can I be a good geologist?	What should I do to be a great geologist?	1
8	17	18	When do you use I instead of l?	When do you use "&" instead of "and"?	0
9	19	20	Motorola (company): Can I hack my Charter Motorola DCX3400?	How do I hack Motorola DCX3400 for free internet?	0

Table 1: Sample records from the Quora Question Pairs dataset.

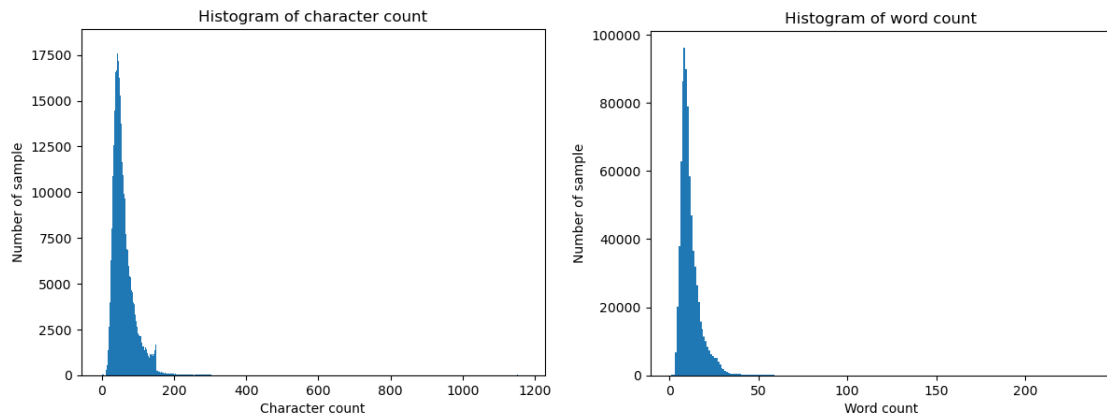
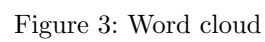


Figure 2: Histograms



3 Literature review

Before we dive into the problem, let's review the 2 important paper that explain how the 2 models we will be using work: BERT and SBERT.

3.1 BERT

BERT (Bidirectional Encoder Representations from Transformers) was introduced in the paper "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" [3] by Devlin et al at Google AI Language in 2019. Unlike other language models, like the original "Attention is all you need" paper (encoder-decoder) or GPT (decoder only), BERT has encoder only architecture, and it was pretrained without future masking, allowing information to flow in 2 direction, left to right and right to left, making the model able to represent better the whole sequence. The pretrained model is then finetuned for various downstream tasks.

There are 2 step in BERT framework: pre-training and fine-tuning. The model is first pre-trained on unlabeled data. The fine-tuning step take the pre-trained BERT model and train the parameters with their respective labeled data.

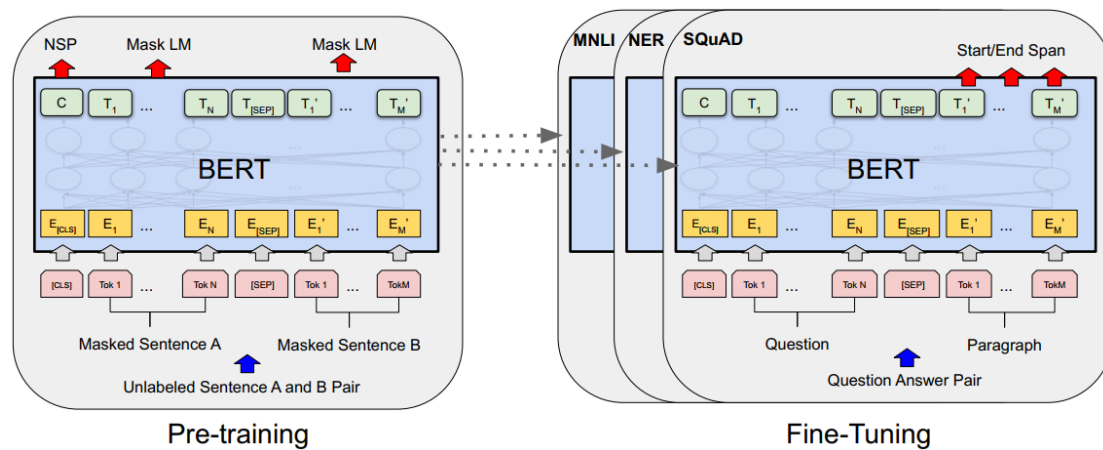


Figure 4: BERT training

BERT's architecture is a multiple transformer encoder layer, with 2 model sizes, BASE (768d hidden size, 110M params) and LARGE (1024d hidden size, 340M params). BERT allow the input to be either a single sentence or a pair of sentence (question-answer, next sentence, etc.). It use WordPiece embedding with vocab size of 30000. The first token is always CLS for classification tasks. If the input is a pair of sentences, there will be a SEP token between them.

BERT was pre-trained on BooksCorpus (800M words) and English Wikipedia (2500M words), with 2 tasks: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP).

- In MLM, random token are masked and the model has to predict them. Because we are predicting the MASK token instead of the next token (like GPT), we can allow attention to flow to both past and future token, allowing bidirectional information
- In NSP, we choose a pair of sentences and have the model predict whether sentence B is the next sentence of sentence A in the corpus. This task help the model understand relationship between sentences and improve downstream task, like Question Answering (QA) and Natural Language Inference (NLI)

For fine-tuning, BERT was trained end-to-end from the pre-trained parameters with the tasks' input and output. At the time it achieved SOTA on 11 tasks:

- General Language Understanding Evaluation (GLUE) [9]:
 - MultiNLI Matched/Mismatched (MNLI-(m/mm))
 - **Quora Question Pairs (QQP)**
 - Question NLI (QNLI)
 - The Stanford Sentiment Treebank (SST-2)
 - The Corpus of Linguistic Acceptability (CoLA)
 - Semantic Textual Similarity Benchmark (STS-B)
 - Microsoft Research Paraphrase Corpus (MRPC)
 - Recognizing Textual Entailment (RTE)
- The Stanford Question Answering Dataset (SQuAD)
 - SQuAD v1.1
 - SQuAD v2.0
- The Situations With Adversarial Generations (SWAG)

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

Table 2: GLUE results

Here we focus on QQR, as this will be the dataset we are working with in this project. The F1 score for QQP of BERT BASE from the paper table was 71.2

3.2 SBERT

BERT is a powerful model for multiple downstream task. But when it come to semantic textual similarity, specifically finding similarity between 2 sentences, it require feeding both sentences into the network pairwise. Which mean, given n sentences, we need to calculate $\frac{n(n-1)}{2}$ combination of sentence pair to get the similarity between each pair, which is inefficient. Another method is to get embedding of each sentence individually, to compute the similarity between the embedding vectors. It's common to use the last hidden state to represent the sentence, such as the **CLS** token, or the average pool of the embeddings. SBERT paper argues that this approach yeild worse embedding result than other existing approaches, like GLoVe.

SBERT (Sentence-BERT) was introduced in the paper "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks" [7] by Nils Reimers et al in 2019. SBERT addressed the mentioned issues by taking BERT and training it on their specific task suitable for encoding the sentence for sementic similarity and clustering tasks.

SBERT add a pooling layer to the output of BERT, with many availble config (**CLS**, **MEAN**, **MAX**), thought the default is **MEAN**, and use cosine similarity as the default similarity function.

SBERT used the following objective function:

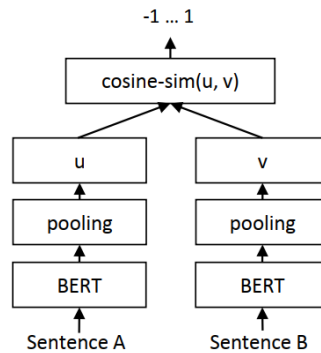


Figure 5: SBERT inference

- **Classification:** With 2 sentence embedding u and v :

$$o = \text{softmax}(W_t(\text{concat}(u, v, |u - v|)))$$

we then use cross entropy loss

- **Regression:** Compute the cosine similarity between 2 embedding, then use mean square error loss
- **Triplet:** Given an anchor sentence a , a positive sentence p , and a negative sentence n , we pull p closer to and n further away from a .

$$\max(\|s_a - s_p\| - \|s_a - s_n\| + \epsilon, 0)$$

SBERT was trained on SNLI and Multi-Genre NLI datasets with Adam optimizer and linear learning rate warm-up. It achieved remarkable results on multiple benchmark. The best selling point for SBERT is its efficiency in encoding sentences with same trade off for accuracy.

4 Training

Considering the binary classification task with dataset ‘Quora Question Pairs’, we will carry out experiment with BERT and SBERT to see how well they perform.

- With BERT, we use the sentence pair approach, which input the 2 sentences into the model at once, then pass the output CLS token through a logistic classifier. We will be training all parameters (BERT + FC layer).
- With SBERT, we use 2 approaches:
 1. We pass each sentence to the model separately, get the 2 embedding u and v , and use cosine similarity to calculate the score. The result is a scala between -1 and 1 , thus we will choose a threshold to predict the binary class of the sentence pair. This approach does not train the model.
 2. We also get 2 embedding u and v like before, but now we concatenate them $(u, v, |u-v|)$ and pass it through a logistic classifier. We will be training all parameters (SBERT + FC layer).

The training notebooks was ran on Google Colab. Each notebook ran for about 1-2 hours on T4 free tier GPU (16GB VRAM). The code use PyTorch and PyTorch Lightning for logic, which let us configure many training options easily, such as logging, early stopping, optimizer, training step, etc.

4.1 BERT

See `source/bert/bert.ipynb` notebook for more details.

```
1 DATASET_NAME = "AlekseyKorshuk/quora-question-pairs"
2 MODEL_NAME = "google-bert/bert-base-uncased"
3 LOG_DIR = "drive/MyDrive/lms/nlp/assignment"
4 BATCH_SIZE = 32
5 NUM_WORKERS = 2
6 D_MODEL = 768
7 LEARNING_RATE = 3e-5
8 MAX_EPOCHS = 3
9 PATIENCE = 10
```

Listing 1: Configs and hyperparams

We load the model and dataset from Hugging Face’s libraries `transformers` and `datasets`. To stay true to the original BERT paper, we will use `google-bert/bert-base-uncased` [5] (although TinyBERT would be lighter for small GPU). As recommended by BERT paper, we will choose learning rate 3×10^{-5} .

We clean the invalid data from our dataset, then tokenize them into token ids. These ids will be used as input for the model. It’s important to use the correct tokenizer for the model. We also make sure to truncate during tokenization, else it might not fit into BERT’s max sequence length.

```
1 ds_cleaned = ds.filter(
2     lambda row: isinstance(row["question1"], str) and isinstance(row["question2"],
3     str)
4 )
5 ds_tokenized = ds_cleaned.map(
6     lambda row: tokenizer(row["question1"], row["question2"], truncation=True),
7     batched=True,
```

```
7     remove_columns=["qid1", "qid2", "question1", "question2"],  
8 )
```

Listing 2: Preprocess data

After that, we split the data to train, validation, and test sets. The final processing step is collating data. We use the available `DataCollatorWithPadding` to pad each sample to the longest sample of the batch for each mini batch, because pytorch tensor required the data to have consistent length.

For training, we use Pytorch Lightning [1], a library that provide us `LightningModule` to organize the model and training step logic, and `Trainer` API for reducing boilerplate training code.

```
1 class BertQuoraModel(L.LightningModule):  
2     def __init__(self, bert, d_model=D_MODEL):  
3         super().__init__()  
4         self.bert = bert  
5         self.d_model = d_model  
6         self.classifier = nn.Linear(d_model, 1)  
7  
8     def forward(self, input_ids, attention_mask, token_type_ids):  
9         bert_output = self.bert(input_ids, attention_mask, token_type_ids)  
10        cls_token = bert_output.pooler_output  
11        logits = self.classifier(cls_token).reshape(-1)  
12        return logits  
13  
14    ...
```

Listing 3: LightningModule model

BERT input are `input_ids` (the position of each token in the vocab), `attention_mask` (masking the padding token), `token_type_ids` (0 and 1, for BERT to tell the first and second sentence in the input)

During training, we log the metrics for comparison after.

4.2 SBERT

See `source/sbert/sbert.ipynb` notebook for more details.

```
1 DATASET_NAME = "AlekseyKorshuk/quora-question-pairs"  
2 MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"  
3 LOG_DIR = "drive/MyDrive/lms/nlp/assignment"  
4 SEED = 42  
5 BATCH_SIZE = 32  
6 NUM_WORKERS = 2  
7 D_MODEL = 384  
8 LEARNING_RATE = 3e-5  
9 MAX_EPOCHS = 3  
10 PATIENCE = 10
```

Listing 4: SBERT config and hyperparams

The configs for SBERT is quite similar to BERT, with the main difference being the hidden size, which is 384 instead of 768. We are using the most downloaded SBERT model from Hugging Face, and it's a tiny version [8], way smaller than BERT.

Preparing the dataset differ somewhat to the BERT pipeline. This is because we are inputting 2 sentences separately instead of combining them like previously.

```
1 def tokenize_row(row):
2     output_1 = tokenizer(row["question1"], truncation=True)
3     output_2 = tokenizer(row["question2"], truncation=True)
4     return {
5         "input_ids_1": output_1["input_ids"],
6         "attention_mask_1": output_1["attention_mask"],
7         "input_ids_2": output_2["input_ids"],
8         "attention_mask_2": output_2["attention_mask"],
9     }
10
11
12 ds_tokenized = ds_cleaned.map(
13     lambda row: tokenize_row(row),
14     batched=True,
15     remove_columns=["qid1", "qid2", "question1", "question2"],
16 )
```

Listing 5: Preprocess data

And therefore we need to use a custom data collator to pad the token ids and masks of each batch correctly

```
1 def data_collator(batch):
2     data_dict = {}
3
4     for field in ("input_ids_1", "input_ids_2"):
5         field_data = [torch.tensor(sample[field]) for sample in batch]
6         data_dict[field] = pad_sequence(
7             field_data,
8             batch_first=True,
9             padding_value=tokenizer.pad_token_type_id,
10        )
11
12    for field in ("attention_mask_1", "attention_mask_2"):
13        field_data = [torch.tensor(sample[field]) for sample in batch]
14        data_dict[field] = pad_sequence(
15            field_data,
16            batch_first=True,
17            padding_value=0,
18        )
19
20    data_dict["is_duplicate"] = torch.tensor([sample["is_duplicate"] for sample in batch])
21    return data_dict
```

Listing 6: Custom data collator for SBERT

As mentioned above, we use 2 approaches for SBERT, the first without training and the second with training.

4.2.1 Approach 1: Cosine similarity - no training

We take the mean of the output embedding to get a single embedding, taking into account the different sequence length of each sample in a batch

```
1 def mean_pooling(last_hidden_state, attention_mask):
2     mask = attention_mask.unsqueeze(-1)
3     return (last_hidden_state * mask).sum(1) / mask.sum(1)
```

Listing 7: Mean pooling for SBERT's output

All that's left is to evaluate the model by taking cosine similarity of the 2 embedding and make prediction. We choose a threshold of 0.7715 for maximum f1-score, based on the official SBERT documentation [2].

4.2.2 Approach 2: Logistic classifier - training

Similar to how BERT was trained, we also use PyTorch Lightning as the framework to define our training logic.

```
1 class SbertQuoraModel(L.LightningModule):
2     def __init__(self, sbert, d_model=D_MODEL):
3         super().__init__()
4         self.sbert = sbert
5         self.d_model = d_model
6         self.classifier = nn.Linear(3 * d_model, 1)
7
8     def mean_pooling(self, last_hidden_state, attention_mask):
9         mask = attention_mask.unsqueeze(-1)
10        return (last_hidden_state * mask).sum(1) / mask.sum(1)
11
12    def forward(self, input_ids_1, attention_mask_1, input_ids_2, attention_mask_2):
13        output_1 = self.sbert(input_ids_1, attention_mask_1)
14        output_2 = self.sbert(input_ids_2, attention_mask_2)
15
16        embed_1 = self.mean_pooling(output_1.last_hidden_state, attention_mask_1)
17        embed_2 = self.mean_pooling(output_2.last_hidden_state, attention_mask_2)
18        embed = torch.concat((embed_1, embed_2, torch.abs(embed_1 - embed_2)), dim
19                             =1)
20
21        logits = self.classifier(embed).reshape(-1)
22        return logits
```

Listing 8: SBERT LightningModule model

The differences to BERT is that we use a mean pooling layer to get the embedding, and we encode the 2 sentences separately.

5 Result and insight

The result are taken from the training notebooks. Table 3 show the result of our training on QQP dataset. These result are evaluated on the test set, which is only a subset of the QQP dataset. The time in the table is the seconds it take to run inference of the test set.

Model	Accuracy	Precision	Recall	F1-score	Time (second)
BERT	0.8969	0.8488	0.8746	0.8561	276
SBERT(1)	0.7740	0.6514	0.8333	0.7312	48
SBERT(2)	0.8921	0.8331	0.8880	0.8536	69

Table 3: Performance comparison of BERT and SBERT models

We can see that BERT has the highest metrics on accuracy, precision, and F1-score, with SBERT(2) coming at a close second place. SBERT(1) being zero shot and not train has the worst result out of the 3. However, SBERT(1) has the fastest inference time of the three, while BERT BASE was many time slower. This comparison is not very fair for BERT either, because we use the Tiny versions of SBERT but not the tiny version of BERT, so this result is just for reference.

Our experiment were not able to meaningfully explore the aspect of finding similar semantic sentences, as the provided dataset already give us pairs of positive/negative questions, so we don't have to find the correct pair, which would emphasize the difference of $\frac{n(n-1)}{2}$ inference issue mention at the start.

Comparing to the BERT paper's f1 score of 0.712 on QQP, we can say that we did achive better numbers, though only on a subset of the dataset.

The logic for drawing graph are in `source/graph.ipynb`. Below in figure 6 and 7 are some graph taken from Tensorboard of the training process of BERT and SBERT, which can be invoke with "`tensorboard -logdir .`".

For BERT, the graph show the training process, and for SBERT, the graph only show the training process for approach 2, because we do not train the model in approach 1.

Both training process were allowed a maximum epoch of 3, however they both early stopped during their second epochs. This suggest that the pretrained model already has strong capability to represent language without needing much fine tuning. Almost all graphs show that the metrics are very noisy during training and functuate rapidly, which suggest that it might need learning rate scheduler, which is currently missing in our training configuration. On the other hand, we might need to test fine tuning only the classifier (linear probing) in the future, which could be faster and might lead to better results.

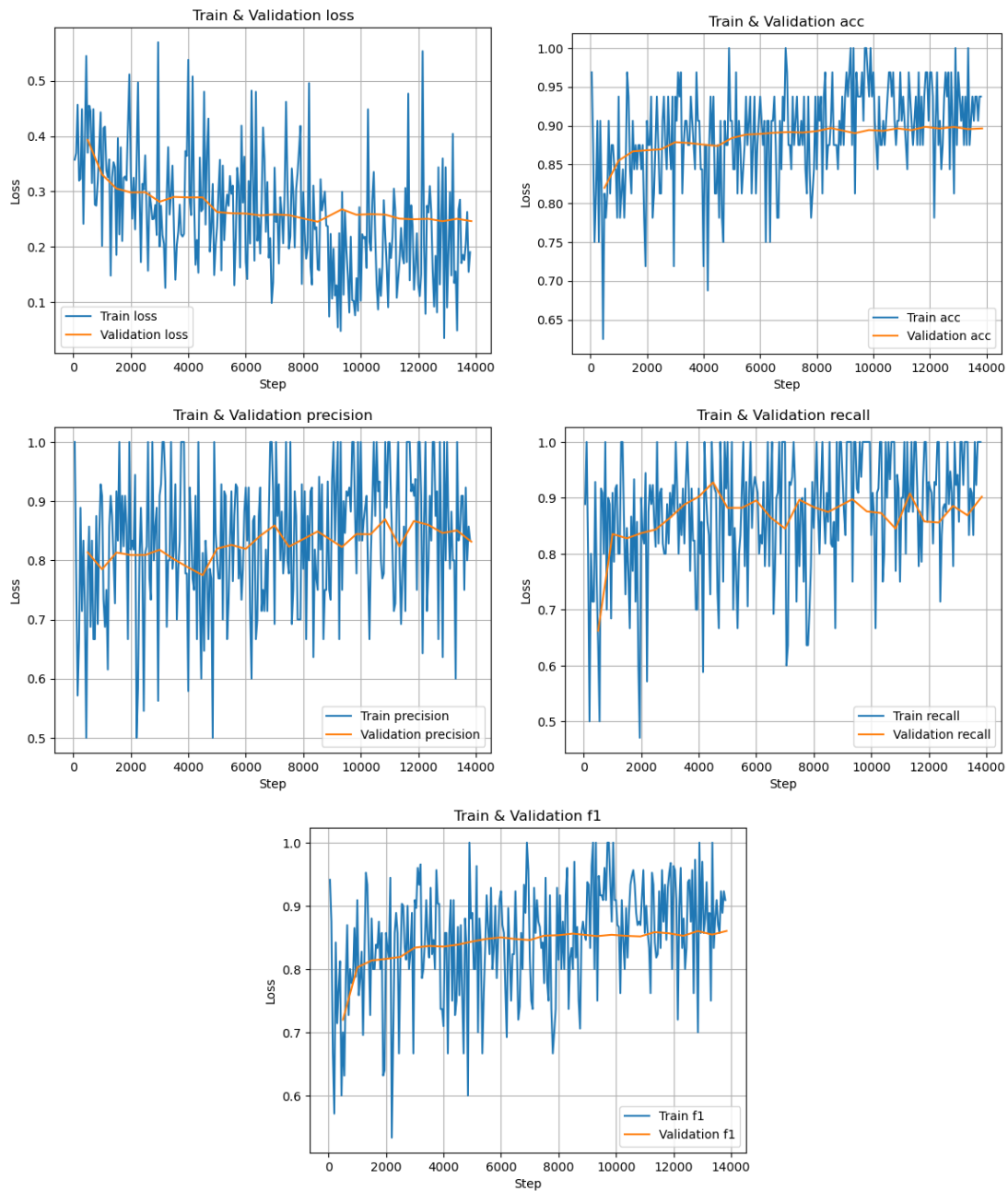


Figure 6: BERT training graph

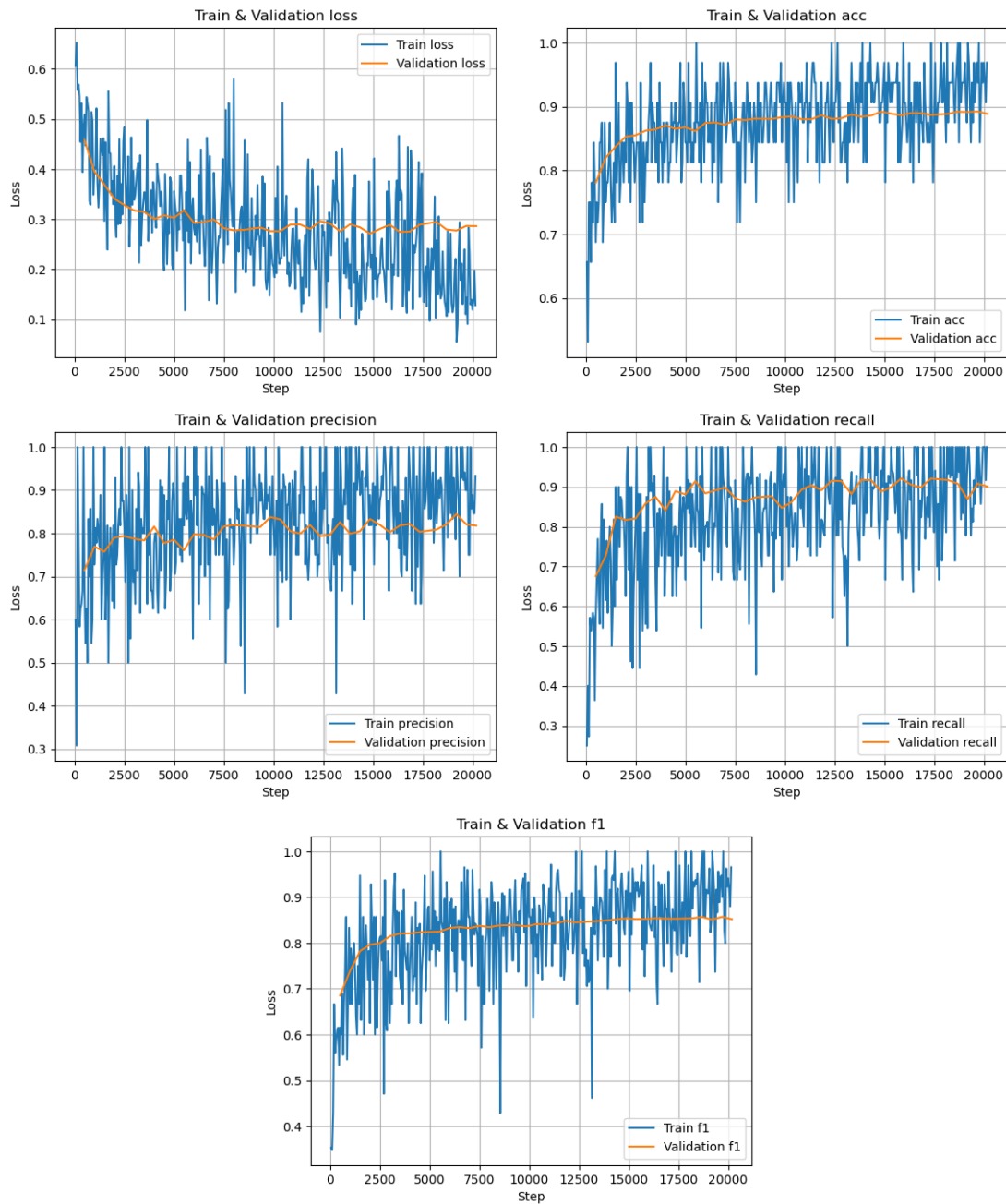


Figure 7: SBERT training graph



6 Conclusion

References

- [1] Lightning AI. *PyTorch Lightning Documentation*. <https://lightning.ai/docs/pytorch/stable/>. Version 2.6.1 documentation. Accessed: 2026-06-14. 2026.
- [2] Sentence Transformers Contributors. *Evaluation — Sentence Transformers Documentation*. https://www.sbert.net/docs/package_reference/sentence_transformer/evaluation.html. Accessed: 2026-06-14. 2026.
- [3] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *arXiv preprint arXiv:1810.04805* (2018). DOI: [10.48550/arXiv.1810.04805](https://doi.org/10.48550/arXiv.1810.04805).
- [4] GeeksforGeeks. *Different Techniques for Sentence Semantic Similarity in NLP*. <https://www.geeksforgeeks.org/nlp/different-techniques-for-sentence-semantic-similarity-in-nlp/>. Last updated: 23 July 2025. Accessed: 2026-06-14. 2025.
- [5] Google and Hugging Face. *BERT Base Model (Uncased)*. <https://huggingface.co/google-bert/bert-base-uncased>. Pre-trained English BERT-base model with 110M parameters using masked language modeling and next sentence prediction. Accessed: 2026-06-14. 2026.
- [6] Kaggle and Quora. *Quora Question Pairs*. <https://www.kaggle.com/c/quora-question-pairs>. Kaggle competition dataset for duplicate question detection. Accessed: 2026-06-14. 2017.
- [7] Nils Reimers and Iryna Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. 2019. DOI: [10.48550/arXiv.1908.10084](https://doi.org/10.48550/arXiv.1908.10084). arXiv: [1908.10084](https://arxiv.org/abs/1908.10084) [cs.CL]. URL: <https://arxiv.org/abs/1908.10084>.
- [8] Sentence Transformers Team. *all-MiniLM-L6-v2*. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>. Sentence embedding model with 22.7M parameters and 384-dimensional embeddings. Accessed: 2026-06-14. 2026.
- [9] Alex Wang et al. *GLUE: General Language Understanding Evaluation Benchmark*. <https://gluebenchmark.com/>. Accessed: 2026-06-14. 2019.